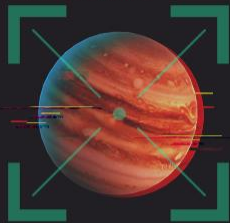
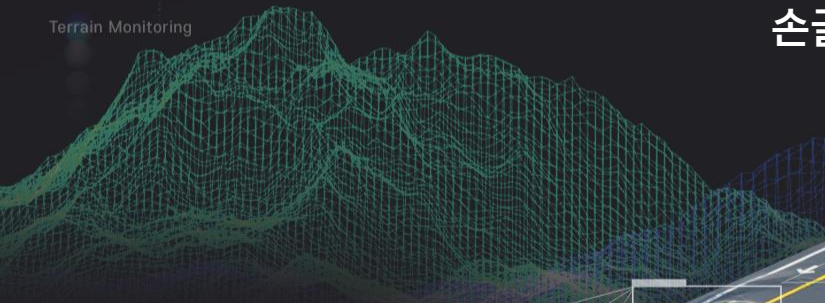


Image Recognition



Terrain Monitoring



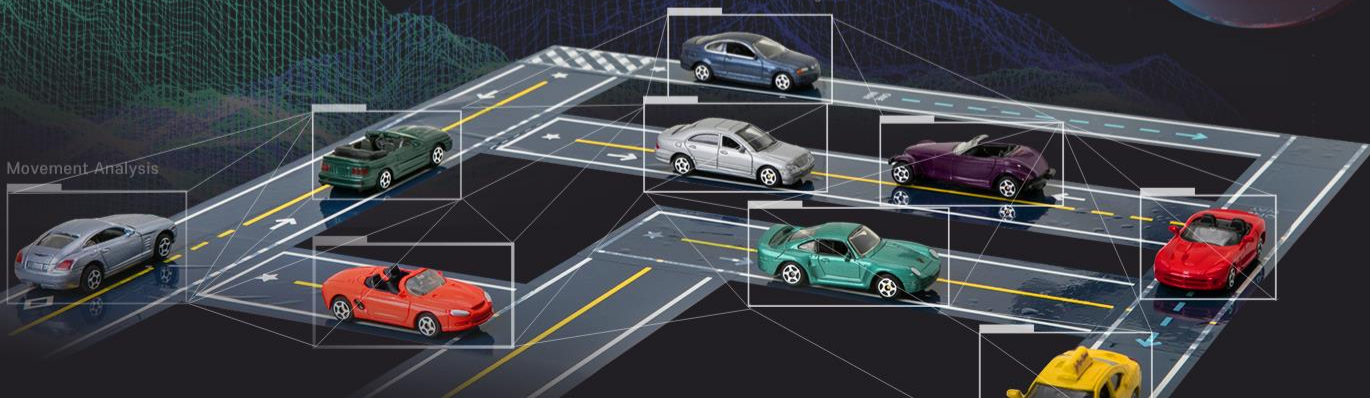
강원혁신플랫폼

컴퓨터 비전

◆
손글씨 숫자 분류 추론

Autonomous Driving

Movement Analysis



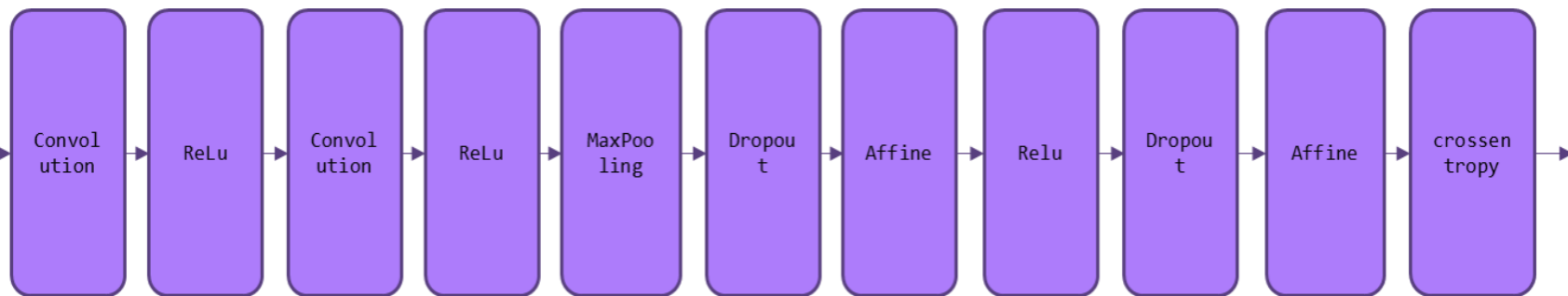


학습된 인공지능망 모델을 읽어와서
테스트 데이터셋으로 모델을 평가 및 추론을 수행해보자

테스트 데이터셋의 25장 이미지로 분류 예측을 수행해 그 결과 이미지를 출력함



- 이를 위해 학습된 모델 “digits_model.h5” 파일을 메모리에 읽어옴
- 앞에서 생성한 4층 인공지능망 네트워크 구조에 학습된 매개변수로 초기화함



인공지능망 네트워크 구성

MNIST 데이터셋을 이용하여 **0~9**까지의 숫자 분류 데이터를 이용해 추론 수행하기

- ▶ 구글 Colab 노트북 화면에서 실행 버튼 클릭
- ▶ 실행 결과 TensorFlow 버전이 '2.12.0'인 것을 알 수 있음



```
1  # tensorflow 버전 확인
2
3  import tensorflow as tf
4
5  tf.__version__
```



'2.12.0'

MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 추론 수행하기

- ➡ 내 구글 드라이브에 연동
- ➡ 구글 Colab 노트북 화면에서 실행 버튼 클릭



```
# 내 구글 드라이브에 연동  
from google.colab import drive
```

```
drive.mount('/content/gdrive')
```



```
Mounted at /content/gdrive
```

MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 추론 수행하기

- ➡ [Google Drive에 연결] > [계정 선택] > [구글 드라이버 접근 허용] 메뉴 선택
- ➡ Google Drive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있음

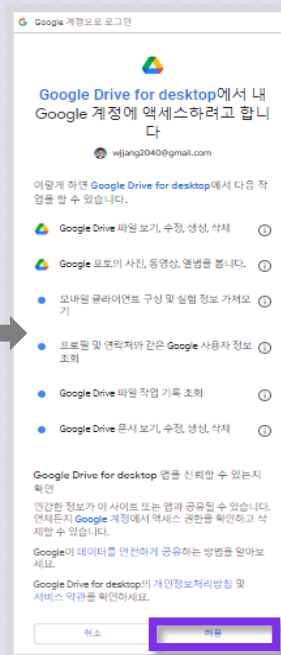
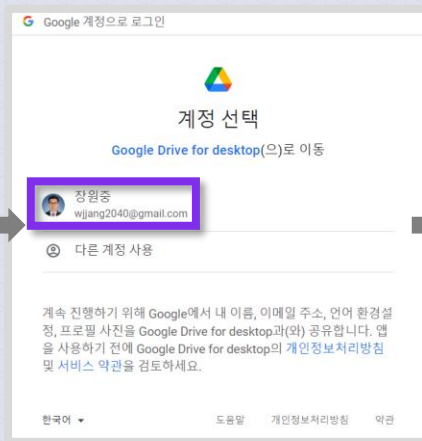
참고 아래와 같이 Google Drive에 연동

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니요

Google Drive에 연결



MNIST 데이터셋을 이용하여 **0~9**까지의 숫자 분류 데이터를 이용해 **추론** 수행하기

🔄 구글 Colab 노트북 화면에서 **실행 버튼** 클릭



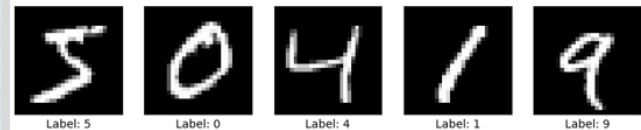
필요로 하는 패키지와 모듈을 불러옵니다.

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
import math
```

MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 추론 수행하기

- ➡ **show_sample()** 함수는 출력할 이미지와 레이블을 넘겨받아 25개 샘플 이미지를 화면에 출력하는 함수를 정의함

```
1 # 25개의 샘플 이미지를 Display 해 주는 함수 정의
2
3 def show_sample(images, labels, sample_count=25):
4
5     # Create a square with can fit {sample_count} images
6     grid_count = math.ceil(math.ceil(math.sqrt(sample_count)))
7     grid_count = min(grid_count, len(images), len(labels))
8
9     plt.figure(figsize=(2*grid_count, 2*grid_count))
10    for i in range(sample_count):
11        plt.subplot(grid_count, grid_count, i+1)
12        plt.xticks([])
13        plt.yticks([])
14        plt.grid(False)
15        plt.imshow(images[i], cmap=plt.cm.gray)
16        plt.xlabel(labels[i])
17    plt.show()
```



MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 추론 수행하기

- ➡ **show_sample_digit()** 함수는 출력할 이미지, 레이블, 특정한 숫자를 넘겨받아 특정한 숫자(Digit)만 25개 이미지를 화면에 출력하는 함수를 정의함

```
1 # 특정한 숫자(Digit)만 25개 Display 해 주는 함수 정의
2
3 def show_sample_digit(images, labels, digit, sample_count=25):
4
5     # Create a square with can fit {sample_count} images
6     grid_count = math.ceil(math.ceil(math.sqrt(sample_count)))
7     grid_count = min(grid_count, len(images), len(labels))
8
9     plt.figure(figsize=(2*grid_count, 2*grid_count))
10    i = 0
11    digit_count = 0
12    while digit_count < sample_count:
13        i += 1
14        if (digit == labels[i]):
15            plt.subplot(grid_count, grid_count, digit_count+1)
16            plt.xticks([])
17            plt.yticks([])
18            plt.grid(False)
19            plt.imshow(images[i], cmap=plt.cm.gray)
20            plt.xlabel(labels[i])
21            digit_count += 1
22    plt.show()
```

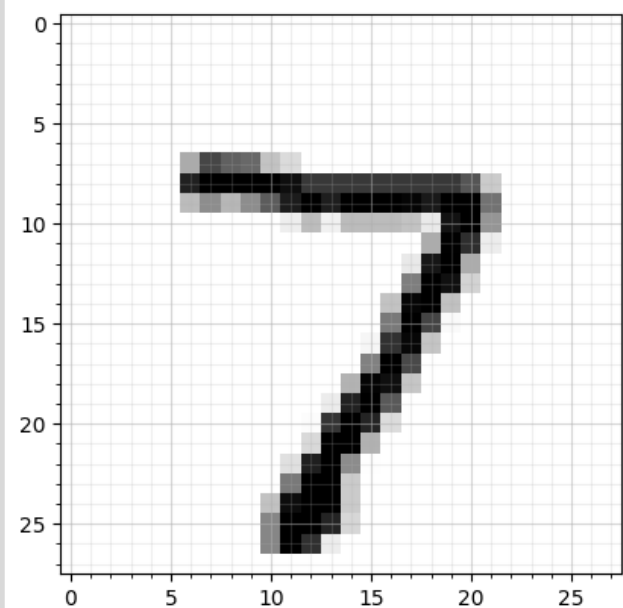


MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 추론 수행하기

- ➡ show_digit_image() 특정한 숫자(Digit)의 이미지와 예측한 Label을 출력하는 함수를 정의함
- ➡ 구글 Colab 노트북 화면에서 실행 버튼 클릭

```

1  # 특정한 숫자(digit)의 이미지와 예측한 label 을 Display 해주는 함수 정의
2
3  def show_digit_image(image):
4      # Draw digit image
5      fig = plt.figure()
6      ax = fig.add_subplot(1, 1, 1)
7
8      # Major ticks every 20, minor ticks every 5
9      major_ticks = np.arange(0, 29, 5)
10     minor_ticks = np.arange(0, 29, 1)
11     ax.set_xticks(major_ticks)
12     ax.set_xticks(minor_ticks, minor=True)
13     ax.set_yticks(major_ticks)
14     ax.set_yticks(minor_ticks, minor=True)
15
16     # And a corresponding grid
17     ax.grid(which='both')
18
19     # Or if you want different settings for the grids:
20     ax.grid(which='minor', alpha=0.2)
21     ax.grid(which='major', alpha=0.5)
22     ax.imshow(image, cmap=plt.cm.binary)
23
24     plt.show()
    
```





■ MNIST 데이터셋 목적

- 🔄 이미지 분류 모델을 Training시키고 Testing하기 위해서 다운로드함

■ MNIST 데이터셋 구성

손으로 작성된
60,000개의 훈련용 이미지

+

10,000개의 테스트 이미지

- 🔄 MNIST 데이터셋에 있는 각각의 이미지들은 단 하나의 숫자(Digit)만 갖고 있음
- 🔄 28×28 그레이스케일(Grayscale) 이미지로 되어있음

MNIST 데이터셋을 이용하여 **0~9**까지의 숫자 분류 데이터를 이용해 추론 수행하기

- ➡ Keras에 있는 MNIST 데이터셋 다운로드
- ➡ 테스트 데이터 10,000개가 있음

```
1 # Keras에 있는 MNIST dataset을 다운로드 받습니다.  
2  
3 mnist = keras.datasets.mnist  
4 (train_images, train_labels), (test_images, test_labels) = mnist.load_data()  
5  
6 print("Test Data :", test_images.shape, test_labels.shape)
```

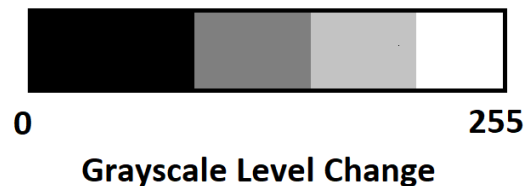
📄 Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 [=====] - 0s 0us/step
Test Data : (10000, 28, 28) (10000,)

MNIST 데이터셋을 이용하여 **0~9**까지의 숫자 분류 데이터를 이용해 추론 수행하기

🔄 테스트 데이터셋의 **0번째 이미지 값**을 출력한 결과, **0~255** 사이의 값으로 구성됨

```
1 # 테스트 데이터 원본 이미지 출력
2
3 print(test_images[0, 0:10, 0:10])
```

```
[[ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0 84 185 159 151]
 [ 0  0  0  0  0  0 222 254 254 254]
 [ 0  0  0  0  0  0 67 114 72 114]]
```



• 테스트 데이터셋의 0번째 이미지 값을 출력한 결과, 0~1 사이의 값으로 구성됨

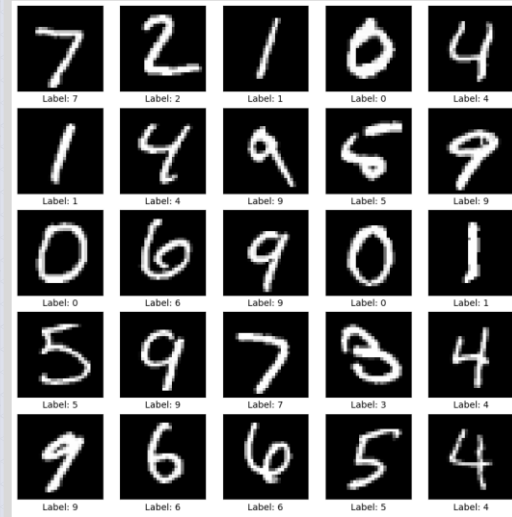
정규화 : $0 \sim 255 \rightarrow 0 \sim 1$

```
(10000, 28, 28)
[[0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      0.
  0.      0.      0.      0.      ]
 [0.32941176 0.7254902 0.62352941 0.59215686]
 [0.      0.      0.      0.      0.      0.
  0.87058824 0.99607843 0.99607843 0.99607843]
 [0.      0.      0.      0.      0.      0.
  0.2627451 0.44705882 0.29235294 0.44705882]]
```

MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 추론 수행하기

- ➡ show_sample() 함수로 테스트용 데이터셋에서 샘플 이미지 25개를 출력함
- ➡ 출력할 이미지와 레이블에 인자 값을 전달함

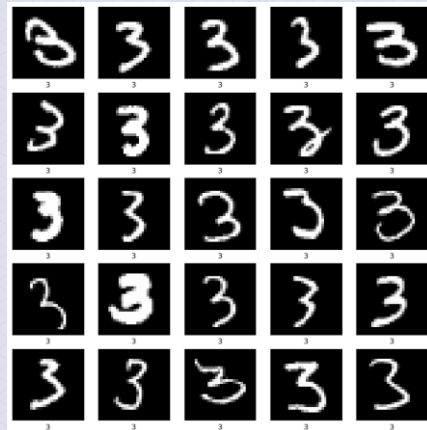
```
1 # show_sample() 함수로 training dataset 에서 25개의 샘플 이미지를 Display 해 줍니다.  
2 # - 이미지와 레이블을 이자값으로 전달  
3  
4 show_sample(test_images, ['Label: %s' % label for label in test_labels])
```



MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 추론 수행하기

- ➡ show_sample_digit() 함수로 테스트 데이터셋에서 특정한 숫자(Digit)만 25개를 출력함
- ➡ 이미지, 레이블, 특정한 숫자(여기서는 숫자 3)을 넘겨받아 특정한 숫자 25개를 출력함

```
1 # show_sample_digit 함수로 training dataset에서 특정한 숫자(Digit)만 25개 Display 해 줍니다.  
2 # - 이미지와 레이블을 이자값으로 전달  
3  
4 show_sample_digit(test_images, test_labels, 3)
```



MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 추론 수행하기

- 저장된 인공신경망 모델을 읽어옴
- 구글 Colab 노트북 화면에서 실행 버튼 클릭

학습된 모델은
Digit Classification 폴더 아래의
“/model/digits_model.h5” 파일에 저장됨

```
1 # 저장해 두었던 모델을 불러옵니다.
2
3 import tensorflow as tf
4
5 model = tf.keras.models.load_model('gdrive/My Drive/CV/Digits Classify/model/digits_model.h5')
6 model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
reshape (Reshape)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 26, 26, 32)	320
conv2d_1 (Conv2D)	(None, 24, 24, 64)	18496
max_pooling2d (MaxPooling2D)	(None, 12, 12, 64)	0
dropout (Dropout)	(None, 12, 12, 64)	0
flatten_1 (Flatten)	(None, 9216)	0
dense (Dense)	(None, 128)	1179776
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1290

Total params: 1,199,882
 Trainable params: 1,199,882
 Non-trainable params: 0



■ 학습된 모델을 테스트 데이터셋으로 평가함

🔄 테스트 데이터셋으로 평가한 모델의 정확도는 99.33% 임

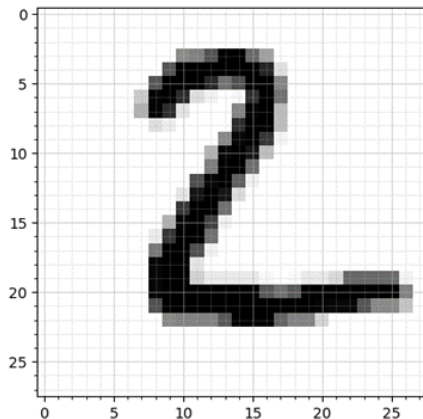
```
[15] 1  # model evaluate : 모델을 평가해 봅니다.  
      2  
      3  test_loss, test_acc = model.evaluate(test_images, test_labels)  
      4  
      5  print('Test accuracy:', test_acc)
```

```
📄 313/313 [=====] - 7s 3ms/step - loss: 0.0257 - accuracy: 0.9933  
Test accuracy: 0.9933000206947327
```


예측 결과 중에서 1번째 숫자의 이미지와 예측한 레이블(Label)을 출력해보자

- show_digit_image() 함수로 특정 이미지의 레이블 출력
- 아래 코드와 테스트 이미지에서 두 번째 이미지의 숫자 레이블이 “2”인 것을 알 수 있음

```
1 # 예측 결과 중에서 특정한 숫자(digit)의 이미지와 예측한 label 을 Display 합니다.
2 # - 두 번째 인덱스의 예측값 출력
3
4 selected_digit = 1
5
6 result = predictions[selected_digit]
7 result_number = np.argmax(result)
8 print('Number is %2i' % (result_number))
9
10 show_digit_image(test_images[selected_digit]) # 특정 이미지의 레이블 출력
```





■ show_sample() 함수로 테스트 데이터셋에서 25장의 샘플을 출력함

- 인자 값으로 테스트 데이터, 테스트 데이터의 예측 레이블을 전달함
- 실행 결과, 매우 잘 예측한 것을 알 수 있음

```
1 # show_sample() 함수로 테스트 데이터의 예측한 결과를 Display 합니다.  
2  
3 show_sample(test_images, ['Predicted: %d' % np.argmax(result) for result in predictions])
```

